Playwright Automation Tutorial

A Complete Guide

25 chapters compiled into one PDF

Snehal Kadiya

+91 9974031480

snehaliteng@gmail.com

https://snehaliteng.github.io/

Copyright & Disclaimer

Please do not print this document unnecessarily. Think of the environment — save paper and trees. Share the digital link instead: https://snehaliteng.github.io/tutorials/playwright-tutorial/

← Back to Tutorials

1. Introduction

Course Expectations

This course takes you from zero to productive with Playwright across 25 chapters. By the end, you will write maintainable end-to-end tests, integrate API testing, handle network interception, set up CI/CD, and use AI agents to generate and heal tests. Each chapter includes code examples, diagrams, and hands-on exercises.

Prerequisites

- Basic familiarity with any programming language (JavaScript helpful but not required)
- Node.js installed (v18+)
- A code editor (VS Code recommended)
- Windows, macOS, or Linux

Setting Up Node.js & VS Code

```
# Download Node.js from https://nodejs.org (LTS version)
# Verify installation
node --version
npm --version

# Install VS Code from https://code.visualstudio.com
```

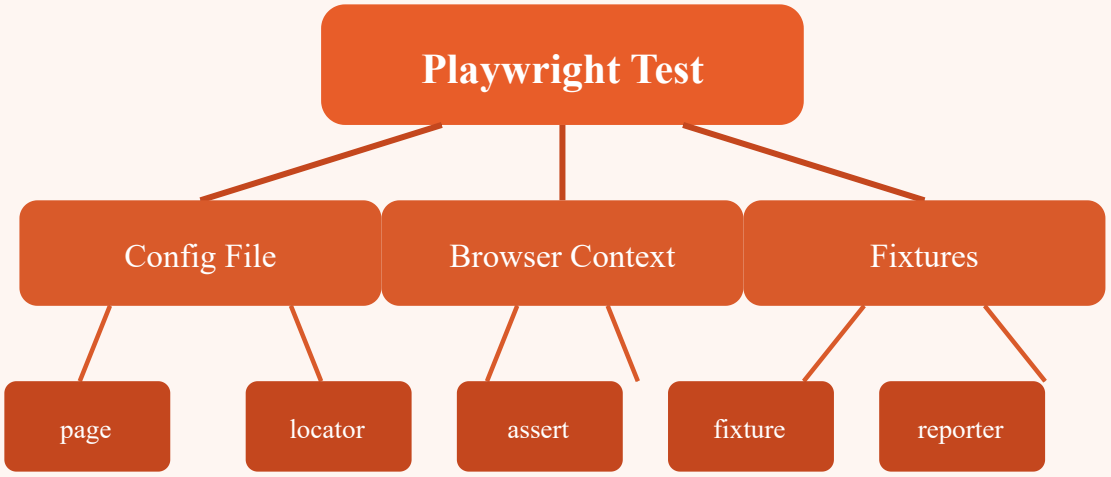
PowerShell Notes (Windows)

On Windows, use PowerShell 5.1+ or PowerShell Core. When running npx commands, you may need to set the execution policy:

```
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
```

First Playwright Project

```
mkdir my-playwright-project
cd my-playwright-project
npm init -y
npm install @playwright/test
npx playwright install
```



Test Runner Architecture

Config → Browser Context → Test Execution → Reports

> 🏏 **Exercise:** Install Node.js, create a new project, install Playwright, and run `npx playwright --version` to verify the setup.

← Back to Tutorials

2. JavaScript & TypeScript Fundamentals

JavaScript Crash Course (3 Hours)

Variables & Data Types

```
// let, const, var
let name = 'Playwright';
const version = 1.48;
var legacy = 'avoid';

// Data types: string, number, boolean, null, undefined, object, array
let isRunning = true;
let count = 42;
let tags = ['e2e', 'api', 'visual'];
```

Functions & Arrow Functions

```
function add(a, b) { return a + b; }
const multiply = (a, b) => a * b;

// Async/await (critical for Playwright)
async function fetchData() {
  const response = await fetch('https://api.example.com');
  return response.json();
}
```

Promises & Async Patterns

```
// Playwright heavily uses async/await
test('example', async ({ page }) => {
  await page.goto('https://example.com');
  await expect(page.locator('h1')).toBeVisible();
});
```

TypeScript Basics

```
// Type annotations
let url: string = 'https://example.com';
let items: number[] = [1, 2, 3];

interface User {
  id: number;
  name: string;
  email?: string; // optional
}

async function getUser(id: number): Promise<User> {
  const res = await fetch(`/api/users/${id}`);
  return res.json();
}
```

💡 **Tip:** You can write Playwright tests in plain JavaScript or TypeScript. TypeScript is recommended for larger projects because of better autocomplete and type safety.

🖍️ **Exercise:** Write an async function that fetches data from a public API (e.g., JSONPlaceholder) and logs the result. Convert it to TypeScript with an interface.

← Back to Tutorials

3. Core Concepts

npm Setup

```
npm init -y
npm install @playwright/test
npx playwright install  # installs Chromium, Firefox, WebKit
```

Test Annotations

```
import { test, expect } from '@playwright/test';

test('basic test', async ({ page }) => {
  await page.goto('https://example.com');
  await expect(page).toHaveTitle(/Example/);
});

test.skip('skipped test', async ({ page }) => { /* ... */ });
test.fixme('needs fix', async ({ page }) => { /* ... */ });
test.only('run only this', async ({ page }) => { /* ... */ });

// Tags
test('smoke test @smoke', async ({ page }) => { /* ... */ });
```

Async/Await in Tests

Every Playwright method returns a Promise. Always use `await`:

```
test('async patterns', async ({ page }) => {
  await page.goto('https://example.com');
  const title = await page.title();
  const url = page.url();
  const text = await page.textContent('h1');
});
```

Browser Contexts

Contexts are isolated browser sessions. Each test gets a fresh context by default:

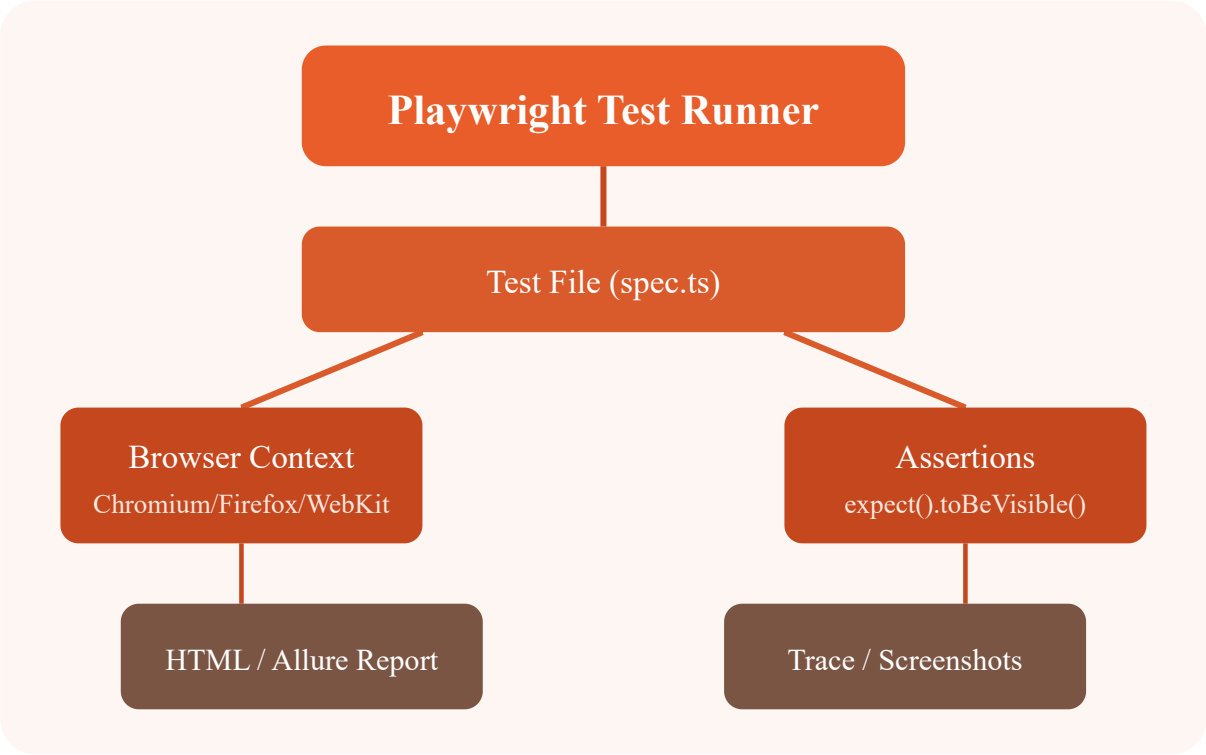
```
test('isolated context', async ({ browser }) => {
  const context = await browser.newContext();
  const page = await context.newPage();
  await page.goto('https://example.com');
  await context.close();
});
```

Config File Basics

```
// playwright.config.ts
import { defineConfig } from '@playwright/test';

export default defineConfig({
  testDir: './tests',
  timeout: 30000,
  retries: 1,
  use: {
    baseURL: 'https://example.com',
    headless: true,
    viewport: { width: 1280, height: 720 },
  },
  projects: [
    { name: 'chromium', use: { browserName: 'chromium' } },
    { name: 'firefox', use: { browserName: 'firefox' } },
  ],
});
```

Assertions

```
import { expect } from '@playwright/test';

await expect(page).toHaveTitle('Dashboard');
await expect(page.locator('.success')).toBeVisible();
await expect(page.locator('button')).toBeEnabled();
await expect(page.locator('input')).toHaveValue('test');
await expect(page.locator('ul li')).toHaveCount(5);
```



🖍️ **Exercise:** Create a Playwright config file with two projects (chromium, firefox), set a base URL, and write a test that navigates to a site and asserts the title.

← Back to Tutorials

4. Basic Methods

Locators

```
// By text
await page.getByText('Login').click();

// By CSS selector
await page.locator('#submit-btn').click();

// By placeholder
await page.getByPlaceholder('Enter email').fill('test@example.com');

// By test ID (recommended)
await page.getByTestId('checkout-button').click();
```

Text Extraction

```
// Get text content
const heading = await page.textContent('h1');
const allText = await page.locator('.item').allTextContents();

// Get input value
const email = await page.inputValue('#email');

// Get attribute
const href = await page.getAttribute('a', 'href');
```

Waits & Auto-Waiting

Playwright auto-waits for elements to be actionable before clicking. You can also use explicit waits:

```
// Auto-waiting (built-in)
await page.getByText('Submit').click();  // waits for visible, enabled, stab

// Explicit waits
await page.waitForSelector('.loading-spinner', { state: 'hidden' });
await page.waitForURL('**/dashboard');
await page.waitForTimeout(2000);  // not recommended
```

Practice Apps

- **SauceDemo** (https://www.saucedemo.com) — e-commerce practice
- **OrangeHRM** (https://opensource-demo.orangehrmlive.com) — HR management
- **TodoMVC** (https://todomvc.com) — simple CRUD

```
// Example: Login to SauceDemo
test('login', async ({ page }) => {
  await page.goto('https://www.saucedemo.com');
  await page.fill('#user-name', 'standard_user');
  await page.fill('#password', 'secret_sauce');
  await page.click('#login-button');
  await expect(page.locator('.inventory_list')).toBeVisible();
});
```

> 🖍️ **Exercise:** Write a test that opens SauceDemo, logs in, extracts the text of all product names, and asserts there are 6 products.

← Back to Tutorials

5. UI Components

Dropdowns

```
// Select by label, value, or index
await page.selectOption('#country', 'India');            // by label
await page.selectOption('#country', { value: 'IN' });   // by value
await page.selectOption('#country', { index: 2 });       // by index

// Multi-select
await page.selectOption('#hobbies', ['Reading', 'Music']);
```

Radio Buttons & Checkboxes

```
// Radio: just click the label or input
await page.getByLabel('Male').check();

// Checkbox
await page.getByRole('checkbox', { name: 'Agree' }).check();
await expect(page.getByRole('checkbox', { name: 'Agree' })).toBeChecked();

// Toggle
await page.getByLabel('Subscribe').uncheck();
```

Child Windows (Popups)

```
// Handle new window/tab
const [newPage] = await Promise.all([
  page.waitForEvent('popup'),
  page.click('#open-new-window'),
]);
await newPage.waitForLoadState();
await expect(newPage).toHaveTitle('New Window');
```

Tabs

```
// Open a new tab in the same context
const page2 = await context.newPage();
await page2.goto('https://example.com');

// Switch between tabs
await page.bringToFront();
await page2.bringToFront();
```

file Uploads

```
// Single file
await page.getByLabel('Upload').setInputFiles('resume.pdf');

// Multiple files
await page.getByLabel('Upload').setInputFiles(['file1.pdf', 'file2.pdf']);

// Remove files
await page.getByLabel('Upload').setInputFiles([]);
```

> 📝 **Exercise:** Write a test that opens a form with dropdown, radio buttons, and checkboxes. Select values, submit, and assert the success message appears.

← Back to Tutorials

6. End-to-End Exercises

Product Purchase Flow

```
test('complete purchase on SauceDemo', async ({ page }) => {
  await page.goto('https://www.saucedemo.com');
  await page.fill('#user-name', 'standard_user');
  await page.fill('#password', 'secret_sauce');
  await page.click('#login-button');

  // Add item to cart
  await page.click('#add-to-cart-sauce-labs-backpack');
  await page.click('.shopping_cart_link');

  // Checkout
  await page.click('#checkout');
  await page.fill('#first-name', 'John');
  await page.fill('#last-name', 'Doe');
  await page.fill('#postal-code', '12345');
  await page.click('#continue');
  await page.click('#finish');

  // Verify order
  await expect(page.locator('.complete-header')).toHaveText('Thank you for y
});
```

Order History Validation

```
test('validate order appears in history', async ({ page }) => {
  // Login and place order
  // Navigate to order history
  await page.click('#menu_button');
  await page.click('text=All Items');
  // ... purchase steps ...

  // Validate order ID visible in history
  await page.click('#menu_button');
  await page.click('text=About');
  // Assert order exists
});
```

Autosuggest Dropdowns

```
test('handle autosuggest dropdown', async ({ page }) => {
  await page.goto('https://example.com/search');
  await page.fill('#search', 'Play');
  await page.waitForSelector('.suggestions');
  await page.getByRole('option', { name: 'Playwright' }).click();
  await expect(page).toHaveURL(/.*Playwright/);
});
```

> 🖍️ **Exercise:** Write a Playwright script to log in to SauceDemo, add an item to cart, complete checkout, and validate the order ID appears in the confirmation page.

← Back to Tutorials

7. Smart Locators

Playwright recommends using accessible locators that mimic how users interact with your app. These are more resilient than CSS/XPath.

GetByRole

```
await page.getByRole('button', { name: 'Submit' }).click();
await page.getByRole('link', { name: 'Home' }).click();
await page.getByRole('heading', { name: 'Dashboard' });
await page.getByRole('textbox', { name: 'Email' }).fill('test@test.com');
await page.getByRole('checkbox', { name: 'Agree' }).check();
```

GetByLabel

```
// Great for form inputs with <label>
await page.getByLabel('First Name').fill('John');
await page.getByLabel('Email address').fill('john@example.com');
await page.getByLabel('Password').fill('secret');
```

GetByText

```
// Match by text content (partial or exact)
await page.getByText('Welcome back').click();
await page.getByText('Logout', { exact: true }).click();
```

GetByPlaceholder & GetByTestId

```
await page.getByPlaceholder('Search products...').fill('laptop');
await page.getByTestId('add-to-cart').click();

// data-testid attributes in your HTML:
// <button data-testid="add-to-cart">Add</button>
```

Calendars / Date Pickers

```
// Fill a date input directly
await page.getByLabel('Date of birth').fill('1990-01-15');

// For custom calendar widgets
await page.getByRole('button', { name: 'Open calendar' }).click();
await page.getByRole('gridcell', { name: '15' }).click();
```

Locator Chaining & Filtering

```
// Chain locators
await page.locator('.form').getByRole('button', { name: 'Submit' }).click();

// Filter
await page.locator('.product').filter({ hasText: 'Laptop' }).getByRole('butt
await page.locator('li').filter({ hasNotText: 'Out of stock' });
```

> 🏏 **Exercise:** Refactor a test that uses CSS selectors to use getByRole, getByLabel, and getByTestId instead. Run it and verify it still passes.

← Back to Tutorials

8. Inspectors & Tools

Playwright Inspector

Interactive debugging tool that steps through your test:

```
# Run test in debug mode
npx playwright test --debug

# Or set PWDEBUG=1
$env:PWDEBUG=1
npx playwright test
```

The Inspector pauses at each action and shows the DOM state, locator suggestions, and actions you can perform.

Codegen (Test Generator)

Record interactions and generate Playwright code automatically:

```
npx playwright codegen https://example.com

# Preset to generate JavaScript
npx playwright codegen --target javascript https://example.com

# Preset to generate TypeScript
npx playwright codegen --target typescript https://example.com
```

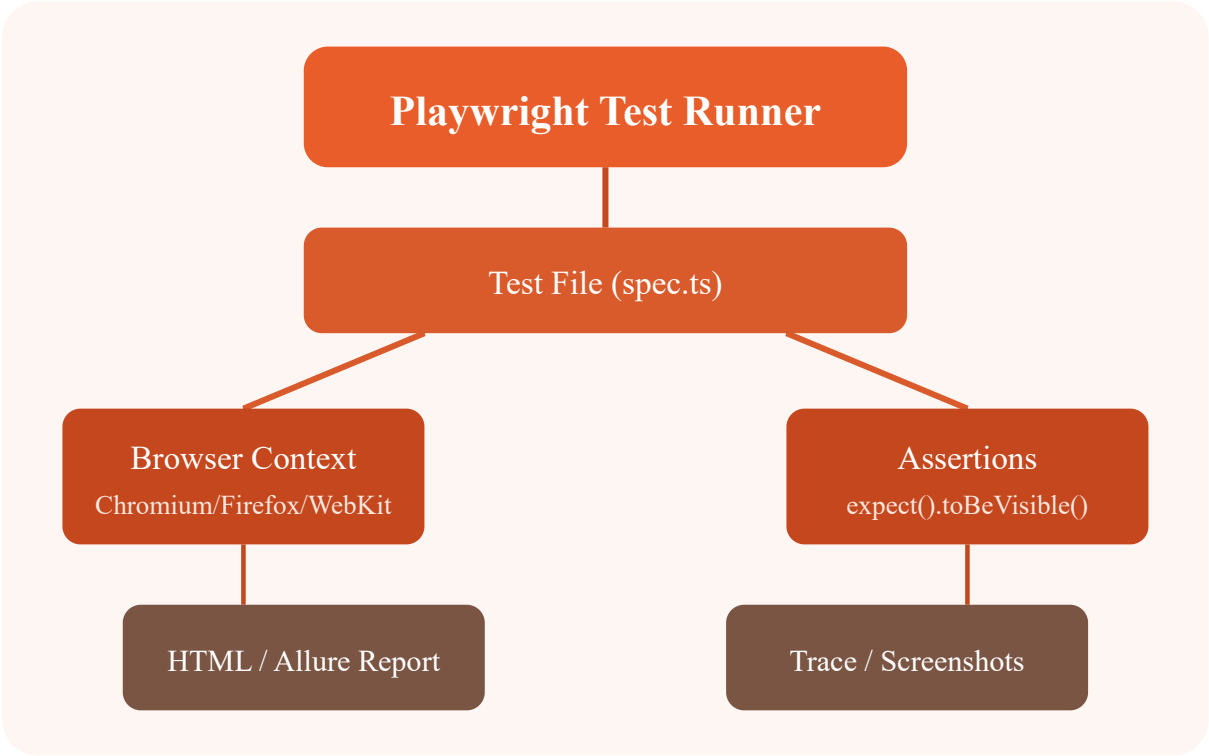
Trace Viewer

View a full trace of your test with DOM snapshots, network logs, and timing:

```
// In config: trace: 'on-first-retry' or 'on'

// View trace
npx playwright show-trace trace.zip
```

HTML Report

```
npx playwright show-report

// Reports are generated in playwright-report/ by default
```



> 🖍️ **Exercise:** Run Codegen on SauceDemo, record a login flow, save the generated script, and run it with the Inspector.

← Back to Tutorials

9. Dialogs & Frames

Alerts, Confirms & Prompts

```
// Accept dialog
page.on('dialog', dialog => dialog.accept());
await page.getByText('Show Alert').click();

// Dismiss dialog
page.on('dialog', dialog => dialog.dismiss());

// Get dialog message
page.on('dialog', dialog => {
  console.log(dialog.message());
  dialog.accept('typed text');  // for prompt dialogs
});
```

Hidden vs Displayed Elements

```
// Check visibility
await expect(page.getByTestId('loading-spinner')).toBeHidden();
await expect(page.getByTestId('success-message')).toBeVisible();

// Force click even if hidden
await page.getByText('Hidden Button').click({ force: true });
```

Handling IFrames

```
// Get frame by name or URL
const frame = page.frame({ name: 'iframe-name' });
// OR
const frame = page.frameLocator('#my-iframe');

// Interact with elements inside the frame
await frame.getByLabel('Name').fill('John');
await frame.getByRole('button', { name: 'Submit' }).click();

// Nested frames
const innerFrame = frame.frameLocator('.nested-frame');
```

Frame Navigation

```
test('iframe navigation', async ({ page }) => {
  await page.goto('https://example.com/iframe-page');
  const frame = page.frameLocator('#content-frame');
  await frame.getByRole('link', { name: 'Next Page' }).click();
  await expect(frame.locator('h1')).toHaveText('Page 2');
});
```

> 🏏 **Exercise:** Find a page with an iframe (e.g., a sandboxed form). Write a test that fills a form inside the iframe and submits it. Handle any dialog that appears.

← Back to Tutorials

10. API Testing

Playwright provides a built-in `APIRequestContext` for making HTTP requests. This lets you combine API and UI tests seamlessly.

Request Methods

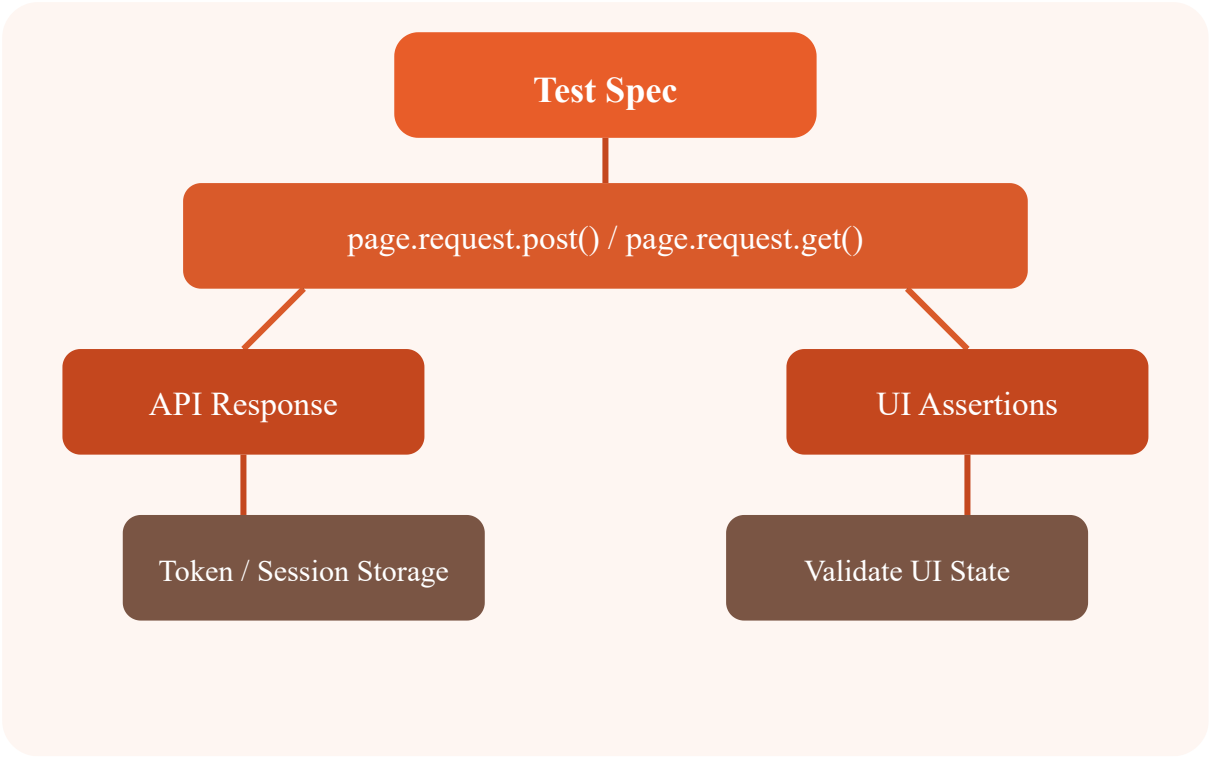
```
import { test, expect } from '@playwright/test';

test('GET request', async ({ request }) => {
  const response = await request.get('https://api.example.com/users');
  expect(response.status()).toBe(200);
  const body = await response.json();
  expect(body.length).toBeGreaterThan(0);
});

test('POST request', async ({ request }) => {
  const newUser = await request.post('https://api.example.com/users', {
    data: { name: 'John', email: 'john@example.com' }
  });
  expect(newUser.status()).toBe(201);
});

test('PUT and DELETE', async ({ request }) => {
  await request.put('https://api.example.com/users/1', {
    data: { name: 'Updated' }
  });
  await request.delete('https://api.example.com/users/1');
});
```

Token Handling

```
test('authenticated API call', async ({ request }) => {
  // Get token
  const auth = await request.post('https://api.example.com/auth/login', {
    data: { username: 'admin', password: 'password' }
  });
  const { token } = await auth.json();

  // Use token
  const response = await request.get('https://api.example.com/protected', {
    headers: { Authorization: `Bearer ${token}` }
  });
  expect(response.status()).toBe(200);
});
```

Utils Refactor

```
// utils/api.ts
import { APIRequestContext } from '@playwright/test';

export async function createUser(request: APIRequestContext, data: any) {
  const response = await request.post('/api/users', { data });
  return response.json();
}

export async function getToken(request: APIRequestContext) {
  const res = await request.post('/api/auth/login', {
    data: { username: 'admin', password: 'password' }
  });
  return (await res.json()).token;
}
```



🏏 **Exercise:** Write a test that creates a user via API, then uses the UI to log in and verify the user's profile page shows the correct name.

← Back to Tutorials

11. Network Interception

Route Method

The `page.route()` method intercepts network requests. You can modify, fulfill, or abort them.

```
// Intercept and modify response
await page.route('**/api/users', async route => {
  const response = await route.fetch();  // allow original request
  const json = await response.json();
  json.push({ id: 999, name: 'Mock User' });
  await route.fulfill({ response, json });
});

// Abort specific requests
await page.route('**/analytics.js', route => route.abort());
await page.route('**/*.png', route => route.abort());
```

Session Storage

```
// Set session storage before navigating
await page.addInitScript(() => {
  window.sessionStorage.setItem('auth_token', 'mock-token-123');
});

// Or after navigation
await page.evaluate(() => {
  sessionStorage.setItem('theme', 'dark');
  localStorage.setItem('preferences', JSON.stringify({ lang: 'en' }));
});
```

Mocking API Responses

```
// Mock entire API response
await page.route('**/api/products', async route => {
  await route.fulfill({
    status: 200,
    contentType: 'application/json',
    body: JSON.stringify([
      { id: 1, name: 'Mock Product', price: 99.99 }
    ])
  });
});

// Network throttling simulation
await page.route('**/*', async route => {
  await new Promise(r => setTimeout(r, 1000));  // add delay
  await route.continue();
});
```

Waiting for Network

```
// Wait for specific response
const response = await page.waitForResponse('**/api/checkout');
const data = await response.json();
expect(data.success).toBe(true);

// Wait for network idle
await page.waitForLoadState('networkidle');
```

> 🏏 **Exercise:** Write a test that intercepts the product list API and returns a mock response with 3 products. Verify the UI shows exactly 3 products.

← Back to Tutorials

12. Visual Testing

Screenshots

```
// Full page screenshot
await page.screenshot({ path: 'screenshots/full-page.png', fullPage: true });

// Element screenshot
await page.locator('.header').screenshot({ path: 'screenshots/header.png' });

// Screenshot with mask (hide dynamic content)
await page.screenshot({
  path: 'screenshots/dashboard.png',
  mask: [page.locator('.user-avatar')]
});
```

Screenshot Assertions (Visual Diff)

```
import { test, expect } from '@playwright/test';

test('visual comparison', async ({ page }) => {
  await page.goto('https://example.com');
  await expect(page).toHaveScreenshot('homepage.png', {
    maxDiffPixels: 100,
    threshold: 0.2,  // 0-1, lower = stricter
  });
});

// Update baseline snapshots
// npx playwright test --update-snapshots
```

Visual Diff Algorithms

Strategy	Description	When to Use
Pixel-by-pixel	Compare every pixel	Strict comparisons
Threshold	Allow small color differences	Cross-browser testing
Max diff pixels	Allow N pixels to differ	Animations, timestamps
Mask	Exclude regions from diff	Dynamic content areas

Config for Visual Testing

```
// playwright.config.ts
use: {
  screenshot: 'only-on-failure',
  // OR
  screenshot: 'on',
}
// For visual tests, set:
expect: {
  toHaveScreenshot: {
    maxDiffPixels: 200,
    stylePath: './visual-overrides.css',
  }
}
```

> 🖍️ **Exercise:** Take a screenshot of a page, make a small CSS change (e.g., change a button color), re-run the visual test, and observe the diff output.

← Back to Tutorials

13. Excel Utils with ExcelJS

Installing ExcelJS

```
npm install exceljs
```

Reading Excel Files

```
import ExcelJS from 'exceljs';

async function readExcel(filePath: string) {
  const workbook = new ExcelJS.Workbook();
  await workbook.xlsx.readFile(filePath);
  const worksheet = workbook.worksheets[0];

  worksheet.eachRow((row, rowNumber) => {
    console.log(`Row ${rowNumber}:`, row.values);
  });
}

// Use in test
test('read test data from Excel', async () => {
  const data = await readExcel('test-data/users.xlsx');
  // Use data for parameterized tests
});
```

Writing Excel Files

```
async function writeExcel(data: any[], filePath: string) {
  const workbook = new ExcelJS.Workbook();
  const sheet = workbook.addWorksheet('Sheet1');

  // Headers
  sheet.columns = [
    { header: 'ID', key: 'id', width: 10 },
    { header: 'Name', key: 'name', width: 30 },
    { header: 'Email', key: 'email', width: 40 },
  ];

  // Data
  data.forEach(item => sheet.addRow(item));

  await workbook.xlsx.writeFile(filePath);
}

test('export test results to Excel', async () => {
  await writeExcel([
    { id: 1, name: 'John', email: 'john@test.com' }
  ], 'output/results.xlsx');
});
```

Uploading Excel Files in Tests

```
test('upload Excel via UI', async ({ page }) => {
  // Create Excel file
  const workbook = new ExcelJS.Workbook();
  // ... build workbook ...
  await workbook.xlsx.writeFile('temp-upload.xlsx');

  // Upload
  await page.getByLabel('Upload File').setInputFiles('temp-upload.xlsx');

  // Assert success
  await expect(page.getByText('File uploaded')).toBeVisible();
});
```

> 🏏 **Exercise:** Create an Excel file with test data, write a Playwright test that reads it, and uses the data to fill a form repeatedly with different values.

← Back to Tutorials

14. Page Object Pattern & Data Parameterization

Page Object Model

```
// pages/LoginPage.ts
import { Page, Locator } from '@playwright/test';

export class LoginPage {
  readonly page: Page;
  readonly usernameInput: Locator;
  readonly passwordInput: Locator;
  readonly loginButton: Locator;

  constructor(page: Page) {
    this.page = page;
    this.usernameInput = page.locator('#user-name');
    this.passwordInput = page.locator('#password');
    this.loginButton = page.locator('#login-button');
  }

  async goto() {
    await this.page.goto('https://www.saucedemo.com');
  }

  async login(username: string, password: string) {
    await this.usernameInput.fill(username);
    await this.passwordInput.fill(password);
    await this.loginButton.click();
  }
}
```

Using Page Objects

```
import { test } from '@playwright/test';
import { LoginPage } from './pages/LoginPage';
import { InventoryPage } from './pages/InventoryPage';

test('standard login flow', async ({ page }) => {
  const login = new LoginPage(page);
  const inventory = new InventoryPage(page);

  await login.goto();
  await login.login('standard_user', 'secret_sauce');
  await inventory.expectLoaded();
});
```

Data Parameterization

```
// Parameterized test with multiple data sets
const users = [
  { username: 'standard_user', password: 'secret_sauce', expected: 'inventor
  { username: 'locked_out_user', password: 'secret_sauce', expected: 'error'
  { username: 'problem_user', password: 'secret_sauce', expected: 'inventory
];

for (const user of users) {
  test(`login as ${user.username}`, async ({ page }) => {
    const login = new LoginPage(page);
    await login.goto();
    await login.login(user.username, user.password);

    if (user.expected === 'error') {
      await expect(page.locator('.error')).toBeVisible();
    } else {
      await expect(page).toHaveURL(/inventory/);
    }
  });
}
```

> ✏️ **Exercise:** Create Page Objects for a checkout flow (CartPage, CheckoutPage, ConfirmationPage). Write a parameterized test that checks out with different user profiles.

← Back to Tutorials

15. Configurations

Project Configs

```
// playwright.config.ts
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({
  testDir: './tests',
  fullyParallel: true,
  forbidOnly: !!process.env.CI,
  retries: process.env.CI ? 2 : 0,
  workers: process.env.CI ? 1 : undefined,
  reporter: 'html',
  use: {
    baseURL: 'https://www.saucedemo.com',
    trace: 'on-first-retry',
    screenshot: 'only-on-failure',
    video: 'retain-on-failure',
  },
  projects: [
    {
      name: 'chromium',
      use: { ...devices['Desktop Chrome'] },
    },
    {
      name: 'firefox',
      use: { ...devices['Desktop Firefox'] },
    },
    {
      name: 'webkit',
      use: { ...devices['Desktop Safari'] },
    },
    {
      name: 'Mobile Chrome',
      use: { ...devices['Pixel 5'] },
    },
    {
```

```
      name: 'Mobile Safari',
      use: { ...devices['iPhone 13'] },
    },
  ],
});
```

Viewport & Device Emulation

```
// Custom viewport
use: {
  viewport: { width: 1440, height: 900 },
}

// Device emulation
use: { ...devices['iPad Pro 11'] }
use: { ...devices['Galaxy S9+'] }
```

SSL & Browser Options

```
use: {
  ignoreHTTPSErrors: true,       // handle self-signed certs
  bypassCSP: true,               // bypass Content Security Policy
  launchOptions: {
    args: ['--disable-web-security'],  // Chromium args
    slowMo: 100,                       // slow down for demo
  },
}
```

Screenshots & Video

```
use: {
  screenshot: 'only-on-failure',  // 'off', 'on', 'only-on-failure'
  video: 'retain-on-failure',     // 'off', 'on', 'retain-on-failure', 'on-f
  trace: 'on-first-retry',        // 'off', 'on', 'retain-on-failure', 'on-f
}
```

> 📝 **Exercise:** Create a config with 3 browser projects and mobile emulation. Set screenshots and video to capture on failure only. Run a test that fails intentionally and inspect the artifacts.

← Back to Tutorials

16. Test Retries & Parallel Execution

Retries

```
// Config-level retries
export default defineConfig({
  retries: 2,  // retry failed tests up to 2 times
  // OR conditional
  retries: process.env.CI ? 3 : 1,
});

// Test-level retry (overrides config)
test('flakey test', async ({ page }) => {
  test.info().annotations.push({
    type: 'issue',
    description: 'https://github.com/org/repo/issues/123',
  });
});
```

Parallel Execution

```
// Config-level
export default defineConfig({
  fullyParallel: true,     // run all tests in parallel across files
  workers: 4,              // number of parallel workers
  // workers: process.env.CI ? 2 : undefined,
});

// File-level (in test file)
import { test } from '@playwright/test';
test.describe.configure({ mode: 'parallel' });  // 'parallel' | 'serial'
```

Serial Execution

```
// When tests depend on each other
test.describe.configure({ mode: 'serial' });

test('step 1 - create user', async ({ page }) => { /* ... */ });
test('step 2 - login as user', async ({ page }) => { /* ... */ });
test('step 3 - delete user', async ({ page }) => { /* ... */ });

// If step 1 fails, steps 2 and 3 are skipped
```

Sharding (CI)

```
// Run across multiple CI machines
# Machine 1
npx playwright test --shard=1/4

# Machine 2
npx playwright test --shard=2/4

# Machine 3
npx playwright test --shard=3/4

# Machine 4
npx playwright test --shard=4/4
```

Best Practices

- Use retries for flaky tests, but fix the underlying issue
- Keep tests independent for reliable parallel execution
- Use `fullyParallel: true` for maximum speed
- Shard in CI for large test suites

✏️ **Exercise:** Configure retries=2 and fullyParallel=true. Run a test suite with 10 tests. Observe the parallel execution speed. Intentionally fail one test and verify it retries.

← Back to Tutorials

17. Reporting & CI/CD

HTML Reporter

```
// playwright.config.ts
reporter: [
  ['html', { outputFolder: 'playwright-report' }],
  ['list'],  // also show in console
]

// View report
npx playwright show-report
```

Allure Reporter

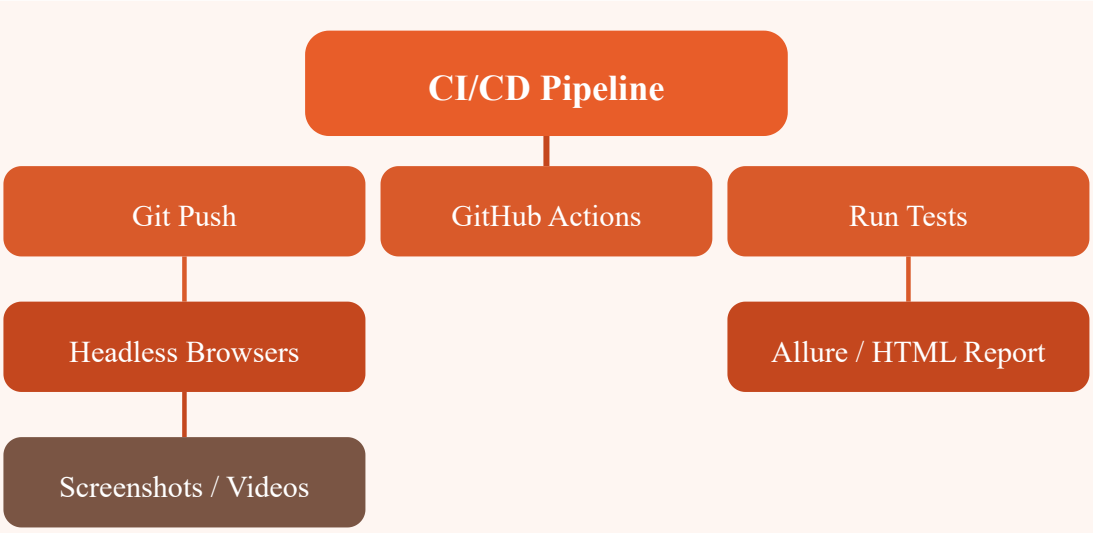
```
npm install @playwright/test allure-playwright

// playwright.config.ts
reporter: [
  ['allure-playwright', { outputFolder: 'allure-results' }],
]

// Generate Allure report
allure generate allure-results -o allure-report --clean
allure open allure-report
```

Jenkins Integration

```
// Jenkins pipeline
pipeline {
  agent any
  stages {
    stage('Install') {
      steps {
        sh 'npm install'
        sh 'npx playwright install --with-deps'
      }
    }
    stage('Test') {
      steps {
        sh 'npx playwright test'
      }
      post {
        always {
          junit 'test-results.xml'
          publishHTML([reportDir: 'playwright-report', reportName: 'Playwrig
        }
      }
    }
  }
}
```

GitHub Actions

```yaml
# .github/workflows/playwright.yml
name: Playwright Tests
on:
  push: { branches: [main] }
  pull_request: { branches: [main] }
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20 }
      - run: npm ci
      - run: npx playwright install --with-deps
      - run: npx playwright test
      - uses: actions/upload-artifact@v4
        if: always()
        with:
          name: playwright-report
          path: playwright-report/
```



> ✏️ **Exercise:** Set up a GitHub Actions workflow that runs Playwright tests on push and uploads the HTML report as an artifact.

← Back to Tutorials

18. TypeScript Refactor

Why TypeScript for Playwright

- Better autocomplete and IntelliSense
- Type safety catches locator/API errors at compile time
- Self-documenting code with interfaces
- Easier refactoring for large test suites

Typing Standards

```
// Properly typed Page Objects
export class InventoryPage {
  constructor(private page: Page) {}

  async addItemToCart(itemName: string): Promise<void> {
    await this.page.getByRole('button', {
      name: `Add ${itemName}`,
    }).click();
  }

  async getCartCount(): Promise<number> {
    const text = await this.page.locator('.cart-badge').textContent();
    return parseInt(text || '0', 10);
  }
}

// Typed fixtures
import { test as base } from '@playwright/test';

interface MyFixtures {
  loggedInPage: Page;
  authToken: string;
}

export const test = base.extend<MyFixtures>({
  loggedInPage: async ({ browser }, use) => {
    const context = await browser.newContext({ storageState: 'auth.json' });
    const page = await context.newPage();
    await use(page);
    await context.close();
  },
  authToken: async ({ request }, use) => {
    const res = await request.post('/api/auth/login', {
      data: { username: 'admin', password: 'pass' }
    });
    const { token } = await res.json();
    await use(token);
```

```
  },
});
```

Refactoring Tests to TypeScript

```
// Before (JavaScript)
test('login', async ({ page }) => {
  await page.fill('#user-name', 'standard_user');
  await page.fill('#password', 'secret_sauce');
  await page.click('#login-button');
});

// After (TypeScript with Page Object)
test('login with POM', async ({ page }) => {
  const loginPage = new LoginPage(page);
  await loginPage.loginAs('standard_user');
  await expect(loginPage.isLoggedIn()).toBeTruthy();
});
```

> 💡 **Tip:** Rename `.js` files to `.ts` and add types incrementally. Playwright's built-in TypeScript support means no separate compilation step.

> 🖍️ **Exercise:** Take a JavaScript test file and rewrite it in TypeScript. Add interfaces for test data and convert Page Objects to typed classes.

← Back to Tutorials

19. Cucumber Integration

Setup

```
npm install @cucumber/cucumber @playwright/test
npm install ts-node typescript  # if using TypeScript
```

Feature Files

```
// features/login.feature
Feature: Login
  As a user
  I want to log into the application
  So that I can access my account

  Background:
    Given I am on the login page

  Scenario: Successful login
    When I enter username "standard_user" and password "secret_sauce"
    And I click the login button
    Then I should see the inventory page

  Scenario: Locked out user
    When I enter username "locked_out_user" and password "secret_sauce"
    And I click the login button
    Then I should see an error message
```

Step Definitions

```
// steps/login.steps.ts
import { Given, When, Then } from '@cucumber/cucumber';
import { Page, Browser, chromium } from '@playwright/test';

let browser: Browser;
let page: Page;

Given('I am on the login page', async function () {
  browser = await chromium.launch();
  const context = await browser.newContext();
  page = await context.newPage();
  await page.goto('https://www.saucedemo.com');
});

When('I enter username {string} and password {string}',
  async (username: string, password: string) => {
    await page.fill('#user-name', username);
    await page.fill('#password', password);
});

When('I click the login button', async () => {
  await page.click('#login-button');
});

Then('I should see the inventory page', async () => {
  await page.waitForSelector('.inventory_list');
  await browser.close();
});
```

Hooks & Tags

```
// hooks.ts
import { Before, After, BeforeAll, AfterAll } from '@cucumber/cucumber';

BeforeAll(async function () {
  // Global setup
});

Before(async function (scenario) {
  // Per-scenario setup
  console.log(`Starting: ${scenario.pickle.name}`);
});

After(async function (scenario) {
  if (scenario.result?.status === 'FAILED') {
    // Take screenshot
  }
});

AfterAll(async function () {
  // Global teardown
});

// Run specific tags
// cucumber-js --tags "@smoke and not @wip"
```

Cucumber Reports

```
// Generate JSON report
// cucumber-js --format json:reports/cucumber-report.json

// Use with Allure or other reporters
npm install allure-cucumberjs
```

> ✏️ **Exercise:** Write a Cucumber feature file for the checkout flow. Implement step definitions using Playwright. Run the tests and generate a report.

← Back to Tutorials

20. DevOps & Cloud

QAOps Principles

QAOps integrates testing into the DevOps pipeline. Tests run on every commit, results are visible in the CI dashboard, and failures block deployments.

Azure Hosting for Test Reports

```
# Deploy Playwright report to Azure Static Web Apps
# .github/workflows/deploy-report.yml
name: Deploy Report
on:
  workflow_run:
    workflows: ["Playwright Tests"]
    types: [completed]

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/download-artifact@v4
        with: { name: playwright-report }
      - name: Deploy to Azure
        uses: Azure/static-web-apps-deploy@v1
        with:
          azure_static_web_apps_api_token: ${{ secrets.AZURE_TOKEN }}
          repo_token: ${{ secrets.GITHUB_TOKEN }}
          action: upload
          app_location: playwright-report
```

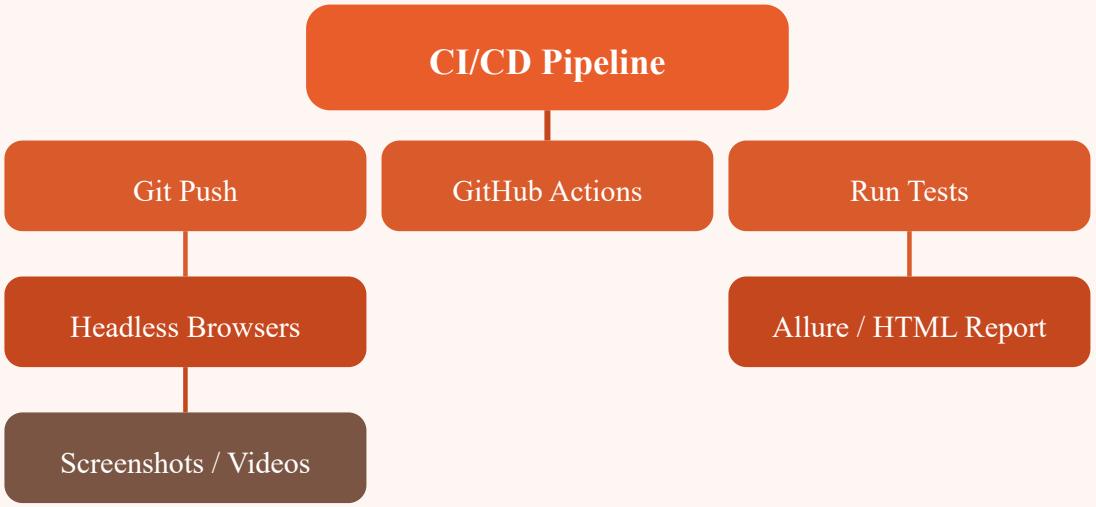
GitHub Actions CI/CD (Full Pipeline)

```yaml
# .github/workflows/ci.yml
name: CI Pipeline
on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20 }
      - run: npm ci
      - run: npx eslint .

  test:
    needs: lint
    runs-on: ubuntu-latest
    strategy:
      matrix:
        project: [chromium, firefox, webkit]
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20 }
      - run: npm ci
      - run: npx playwright install --with-deps
      - run: npx playwright test --project=${{ matrix.project }}
      - uses: actions/upload-artifact@v4
        if: always()
        with:
          name: report-${{ matrix.project }}
          path: playwright-report/

  deploy:
```

```
    needs: test
    runs-on: ubuntu-latest
    steps:
      - run: echo "Deploying to production..."
```

Running Playwright in Docker

```
# Dockerfile
FROM mcr.microsoft.com/playwright:v1.48-jammy
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci
COPY . .
CMD ["npx", "playwright", "test"]
```



🏏 **Exercise:** Create a GitHub Actions workflow that runs Playwright tests across 3 browsers, uploads screenshots on failure, and deploys the HTML report to Azure Static Web Apps.

← Back to Tutorials

21. Playwright Agents & MCP

MCP (Model Context Protocol)

MCP is an open protocol for connecting AI models to tools and data sources. Playwright MCP servers let AI agents control browsers, generate tests, and debug failures autonomously.

Copilot Integration

```
// GitHub Copilot can generate Playwright tests from natural language
// Example prompt in VS Code:
// "Write a Playwright test that logs into SauceDemo and adds an item to car

// Copilot generates:
test('add item to cart', async ({ page }) => {
  await page.goto('https://www.saucedemo.com');
  await page.fill('#user-name', 'standard_user');
  await page.fill('#password', 'secret_sauce');
  await page.click('#login-button');
  await page.click('#add-to-cart-sauce-labs-backpack');
  await expect(page.locator('.shopping_cart_badge')).toHaveText('1');
});
```

Planner Agent

```
// An AI agent that plans test scenarios
// Prompt: "Plan E2E tests for an e-commerce checkout flow"

const plan = {
  scenarios: [
    { name: 'Happy path', steps: ['Login', 'Search product', 'Add to cart',
    { name: 'Invalid coupon', steps: ['Login', 'Add to cart', 'Apply invalic
    { name: 'Empty cart', steps: ['Login', 'Go to cart', 'Verify empty messa
    { name: 'Guest checkout', steps: ['Add to cart', 'Proceed as guest', 'Fi
  ]
};
```

Generator Agent

```
// An AI agent that converts plain English to Playwright code
// Input: "Click the login button and wait for the dashboard"
// Output:
async function clickLoginAndWait(page: Page) {
  await page.getByRole('button', { name: 'Login' }).click();
  await page.waitForURL('**/dashboard');
  await expect(page.getByText('Welcome')).toBeVisible();
}
```

Healing Agent

```
// Auto-healing locators when selectors break
// If a locator fails, the healing agent tries alternative strategies:
// 1. Try getByRole with different accessible name
// 2. Try getByText with partial match
// 3. Try nearby element + relative locator
// 4. Try data-testid fallback
// 5. Report failure with suggested fix
```

> ✏️ **Exercise:** Use GitHub Copilot to generate a Playwright test for a multi-step form. Then use the MCP Playwright server to run a browser command via an AI prompt.

← Back to Tutorials

22. Playwright CLI with Claude Code

Skills.md for Playwright

Create a Skills.md file that teaches Claude how to write Playwright tests:

```
# Playwright Testing Skills

## Test Structure
- Use `@playwright/test` framework
- Import: `import { test, expect } from '@playwright/test'`
- Use async/await for all browser interactions

## Locator Strategy (in priority order)
1. `getByRole()` — accessible locators
2. `getByLabel()` — form inputs
3. `getByTestId()` — data-testid attributes
4. `getByText()` — visible text content
5. `locator()` — CSS selectors (last resort)

## Assertions
- `await expect(page).toHaveTitle()`
- `await expect(locator).toBeVisible()`
- `await expect(locator).toHaveText()`
- `await expect(locator).toHaveValue()`

## Patterns
- Use Page Object model for reusable components
- Use `page.route()` for network mocking
- Use `test.use({ storageState })` for authentication
```

Claude Code Demo

```
# Example Claude interaction
# User: "Write a Playwright test for logging into SauceDemo"

# Claude (using Skills.md context):
# Let me write a test following the established patterns...

import { test, expect } from '@playwright/test';

test('login to SauceDemo', async ({ page }) => {
  await page.goto('https://www.saucedemo.com');
  await page.getByTestId('username').fill('standard_user');
  await page.getByTestId('password').fill('secret_sauce');
  await page.getByTestId('login-button').click();
  await expect(page.getByTestId('inventory-container')).toBeVisible();
});
```

AI Agent Demo: Test Generation

```
# 1. Claude reads Skills.md to understand conventions
# 2. User describes a test scenario in natural language
# 3. Claude generates Playwright code following project patterns
# 4. User runs the test with `npx playwright test`
# 5. If test fails, Claude reads the error and fixes it

# CLI integration
claude "Generate a Playwright test for searching products on example.com"
```

> 🏏 **Exercise:** Create a Skills.md for your Playwright project. Ask Claude Code to generate a test for a checkout flow. Run the generated test and iterate on the result.

← Back to Tutorials

23. Interview Prep (75 mins JavaScript Q&A)

JavaScript Core

1. **What is the difference between `let`, `const`, and `var`?**
   var has function scope, let and const have block scope. const cannot be reassigned.
2. **Explain closures with an example.**
   A closure is a function that remembers its outer variables even after the outer function returns.
3. **What is the event loop?**
   The event loop handles async operations. Callbacks/microtasks (Promise) go to microtask queue; setTimeout/IO go to macrotask queue.
4. **What does `this` refer to in different contexts?**
   In a method: the object. In a function (non-strict): global. In arrow function: lexical scope.

Playwright-Specific

5. **How does Playwright auto-wait work?**
   Playwright waits for elements to be visible, enabled, and stable before actions. No explicit wait needed.
6. **What are the advantages of getByRole over CSS selectors?**
   Accessibility-first, more resilient to DOM changes, better error messages, matches user perception.
7. **Explain browser contexts.**
   Isolated browser sessions with separate storage, cookies, and cache. Each context is like a separate incognito window.
8. **How would you test a file download?**
   Use `page.waitForEvent('download')` and `download.saveAs()`.

9. **What is the difference between `page.route()` and `page.waitForResponse()`?**
   route() intercepts and modifies requests. waitForResponse() waits for a specific response to complete.
10. **How do you run tests in parallel?**
    Set `fullyParallel: true` in config or `test.describe.configure({ mode: 'parallel' })`.

SDET & Automation

11. **What is the Page Object Model?**
    A design pattern that encapsulates page-specific elements and actions in classes for maintainability.
12. **How do you handle dynamic content?**
    Use `waitForSelector`, `waitForURL`, or assertions with auto-retrying (`toBeVisible`).
13. **How do you set up CI for Playwright?**
    GitHub Actions with matrix strategy across browsers. Install with `--with-deps` flag.
14. **What is visual testing and when would you use it?**
    Comparing screenshots pixel-by-pixel. Useful for UI regression, cross-browser consistency, responsive layout checks.
15. **Explain retries and flaky tests.**
    Retries re-run failed tests. Flaky tests pass intermittently — fix root cause rather than relying on retries.

> 🏏 **Exercise:** Practice answering all 15 questions aloud within 75 minutes. Write a Playwright test for the scenario described in question 8 (file download).

← Back to Tutorials

24. Appendix: JS Fundamentals from Scratch

Data Types & Type Coercion

```
// Primitive types: string, number, boolean, null, undefined, symbol, bigint
typeof 'hello';   // 'string'
typeof 42;        // 'number'
typeof true;      // 'boolean'
typeof undefined; // 'undefined'
typeof null;      // 'object' (historical bug)

// Type coercion
'5' - 2;   // 3
'5' + 2;   // '52'
+'5';      // 5 (unary plus)
```

Objects & Arrays

```
// Object
const user = { name: 'John', age: 30 };
user.name;           // 'John'
delete user.age;
Object.keys(user);   // ['name']

// Array
const arr = [1, 2, 3];
arr.push(4);         // [1, 2, 3, 4]
arr.map(x => x * 2); // [2, 4, 6, 8]
arr.filter(x => x > 1); // [2, 3, 4]
arr.find(x => x === 3); // 3
```

Functions, Scope & Hoisting

```
// Function declaration (hoisted)
function greet(name) { return `Hello ${name}`; }

// Function expression (not hoisted)
const greet2 = function(name) { return `Hello ${name}`; };

// Arrow function
const greet3 = (name) => `Hello ${name}`;

// Hoisting
console.log(x); // undefined (not ReferenceError)
var x = 5;
```

Promises & Async/Await (Deep Dive)

```
// Promise chain
fetch('/api/data')
  .then(res => res.json())
  .then(data => console.log(data))
  .catch(err => console.error(err));

// Async/await
async function loadData() {
  try {
    const res = await fetch('/api/data');
    const data = await res.json();
    return data;
  } catch (err) {
    console.error('Failed:', err);
  }
}

// Promise.all (parallel execution)
const [users, products] = await Promise.all([
  fetch('/api/users').then(r => r.json()),
  fetch('/api/products').then(r => r.json()),
]);
```

Destructuring & Spread

```
// Object destructuring
const { name, age } = user;

// Array destructuring
const [first, second] = arr;

// Spread operator
const clone = { ...user, role: 'admin' };
const combined = [...arr, 5, 6];
```

Modules (ESM)

```
// math.js
export const add = (a, b) => a + b;
export default class Calculator { /* ... */ }

// app.js
import Calculator, { add } from './math.js';
```

> 📝 **Exercise:** Write a utility module with async functions that fetch data from a public API, transform it with map/filter, and export the results. Import and test it.

← Back to Tutorials

25. Bonus Lecture: Final Wrap-Up

What You've Learned

- Setting up Playwright with Node.js, VS Code, and TypeScript
- Core concepts: browser contexts, async/await, assertions, fixtures
- Smart locators: getByRole, getByLabel, getByTestId, getByText
- UI components: dropdowns, radio buttons, child windows, tabs, frames
- End-to-end exercises: purchase flow, order history, autosuggest
- API testing with request context, token handling, utils refactoring
- Network interception: route, fulfill, abort, session storage
- Visual testing: screenshots, diff algorithms, masking
- Excel data-driven testing with excelJS
- Page Object Model and data parameterization
- Configurations: projects, viewport, SSL, mobile emulation
- Retries, parallel execution, and sharding
- Reporting: HTML, Allure, Jenkins, GitHub Actions
- TypeScript refactoring and typed fixtures
- Cucumber BDD integration (feature files, step defs, hooks)
- DevOps: Azure hosting, Docker, CI/CD pipelines
- AI agents: MCP, Copilot, planner, generator, healing agents
- Claude Code integration with Skills.md
- Interview preparation with 15 common questions

Next Steps

1. **Build a real project** — Automate a real web app end-to-end
2. **Contribute to open source** — Add tests to your favorite project
3. **Get certified** — Prepare for SDET automation roles

4. **Explore advanced topics** — Playwright Component Testing, WebSocket mocking, Performance testing
5. **Join the community** — Playwright Discord, GitHub discussions, Stack Overflow

Final Challenge

Build a complete automation framework for any web app of your choice with:

- Page Objects for all pages
- API + UI hybrid tests
- Visual regression tests
- CI/CD pipeline with GitHub Actions
- Allure reporting
- TypeScript throughout
- MCP agent integration for self-healing locators

> 🖍️ **Exercise:** Complete the final challenge above. Share your framework on GitHub and link it in your portfolio. Congratulations on finishing the course!